The memory feature moved the memory problem.

Claude can hold a conversation briefly. The application still needs the complete ledger when that five-minute state disappears.

THE STATE CHANGES LOCATION, NOT OWNERSHIP



The shortcut expires. The ledger cannot.

Message Threads are beta. Live latency, token savings and accepted-task economics remain unmeasured.

> **Less context in transit does not mean less state to own.**

Two copies.
Two different jobs.

The server thread carries recent context efficiently. The application ledger makes the conversation recoverable.

SERVER THREAD

Fast path

- Receives the complete first conversation
- Later turns can send only new messages
- Expires five minutes after the last request

APPLICATION LEDGER

Recovery path

- Preserves the ordered conversation
- Survives the thread's expiry boundary
- Replays the complete history into a new thread

The SDK helper keeps history in the running process. Process-restart recovery requires application persistence outside that helper.

> **Temporary state accelerates the healthy path. Durable state owns the failure path.**

Expiry turns a delta back into a full replay.

The normal path sends less. The recovery path reintroduces the complete conversation from the application's own record.



A 404 thread_not_found can also reflect other documented availability failures. Recovery still begins from the application's history.

Design the replay before relying on the shortcut.

One expiry.
One accepted local result.

A deterministic fake service waited past its 300-second lifetime, then forced the client to rebuild the thread from its ledger.

17 + 5 after a 360-second pause

22 accepted · 1 replay · 0 duplicate user turns

Thread lifetime → 300 seconds

Provider calls → 0

Live service evidence → Not measured

> **The local replay preserved both turns once and returned 22.**

A recoverable agent needs more than chat history.

Replaying words restores context. It does not establish whether an external action already happened.

01

1 · Conversation

Persist ordered user, assistant and tool messages.

02

2 · Thread reference

Treat the remote ID as replaceable, short-lived state.

03

3 · Effect ledger

Record whether each external action was accepted or remains unknown.

04

4 · Replay rule

Rebuild context without silently repeating completed effects.

05

5 · Readback

Verify the destination outcome before calling recovery complete.

No external effects were tested; production deduplication remains UNKNOWN.

Pair the conversation ledger with receipts for effects and acceptance.

The mechanism is documented. The economics are not.

Current evidence supports the recovery contract and one local replay. It does not establish live savings or quality.

SUPPORTED NOW

Contract + local replay

- Create, continue, expiry and replay are documented
- One injected expiry recovered locally
- Application source-of-truth duty is explicit

STILL UNKNOWN

Live outcome

- Request-byte and first-token savings
- Recovery tokens, cost and model quality
- Accepted-task cost across real agent loops

Anthropic's release note states smaller requests and lower time to first token qualitatively. No representative live loop was measured here.

> **Measure accepted work, not only smaller requests.**

Use the thread as a shortcut. Treat the ledger as truth.

Temporary server context can reduce repeated transport. Durable application state makes the conversation rebuildable after the shortcut ends.

HEALTHY PATH

How little context can the next request send?

Can context recover without repeating an effect?

Test the five-minute boundary, process loss and one real tool effect before calling the agent recoverable.

Documented beta · One local injection · Live economics UNKNOWN

> **Own the state that recovery requires.**